



UNIVERSIDAD AGRARIA DEL ECUADOR

FACULTAD DE CIENCIAS AGRARIAS
CARRERA DE COMPUTACIÓN EN LÍNEA

PROGRAMACIÓN 2

Especificación del Proyecto
Sistema de Gestión de Biblioteca Universitaria

Docente: Ing. Bryan Vélez

Grupo: #1

Integrantes: Henry Pazmiño · Jodie Parrales · Cindy Ayovi · César Gonzales · Mayura Vera

Semestre: 3SA

Fecha: Julio 2026

1. Introducción

Las bibliotecas universitarias enfrentan el desafío de gestionar eficientemente sus recursos bibliográficos: registro de socios, control de préstamos, devoluciones y reservas. Actualmente, muchas bibliotecas pequeñas y medianas aún utilizan métodos manuales o sistemas genéricos que no se adaptan a sus necesidades específicas.

El presente proyecto tiene como objetivo desarrollar un **Sistema de Gestión de Biblioteca Universitaria** que automatice los procesos diarios de una biblioteca física. El sistema permitirá registrar socios (estudiantes y docentes), administrar el catálogo de libros, controlar préstamos y devoluciones, y gestionar reservas de ejemplares no disponibles.

El sistema se desarrolla como proyecto final de la asignatura **Lenguaje de Programación 2**, aplicando los conceptos de Programación Orientada a Objetos, estructuras de datos, algoritmos de ordenamiento y búsqueda, bases de datos SQLite e interfaz gráfica con Tkinter, todos enseñados durante el semestre por el Ing. Bryan Vélez.

2. Justificación

La Universidad Agraria del Ecuador, en su modalidad en línea, requiere que los estudiantes demuestren competencias en el desarrollo de software aplicando los principios de la programación estructurada y orientada a objetos. Este proyecto integra los conocimientos adquiridos durante el semestre en un producto funcional.

La automatización de una biblioteca universitaria permite:

- **Reducir errores humanos** en el registro manual de préstamos y devoluciones
- **Agilizar la búsqueda** de libros en el catálogo
- **Mantener un control preciso** de los ejemplares disponibles
- **Mejorar la experiencia** del usuario (socios y bibliotecarios)

Además, el proyecto permite aplicar conceptos fundamentales de la ingeniería de software como el modelo incremental, el control de versiones con Git/GitHub, y el uso de documentación técnica.

3. Objetivos

3.1 Objetivo General

Desarrollar un sistema de escritorio para la gestión de una biblioteca universitaria que permita administrar socios, libros, préstamos y reservas, aplicando los principios de la Programación Orientada a Objetos, estructuras de datos, algoritmos de búsqueda y ordenamiento, y persistencia en base de datos SQLite.

3.2 Objetivos Específicos

1. Implementar un modelo de clases utilizando herencia, encapsulamiento, polimorfismo y abstracción para representar los actores del sistema (Persona → Estudiante/Docente, Libro, Préstamo, Reserva).
2. Utilizar estructuras de datos lineales (lista enlazada, cola FIFO, pila LIFO) para la gestión dinámica de la información en memoria.
3. Implementar algoritmos de ordenamiento (burbuja, inserción, merge sort, quick sort) y búsqueda (lineal, binaria) para la manipulación del catálogo.
4. Persistir los datos en una base de datos SQLite con operaciones CRUD completas y claves foráneas.
5. Desarrollar una interfaz gráfica de usuario (GUI) utilizando Tkinter para la interacción con el sistema.

4. Alcance

4.1 Funcionalidades incluidas

- **Gestión de Socios:** registro, consulta y actualización de datos de estudiantes y docentes
- **Gestión de Libros:** alta, consulta y eliminación de libros en el catálogo
- **Préstamos:** realización de préstamos y registro de devoluciones
- **Reservas:** creación de reservas para libros no disponibles (cola de espera FIFO)
- **Búsqueda:** búsqueda de libros por título, autor, y aplicación de búsqueda binaria
- **Ordenamiento:** ordenamiento del catálogo por diferentes criterios usando múltiples algoritmos

4.2 Funcionalidades excluidas

- No incluye módulo de multas o sanciones
- No incluye generación de reportes estadísticos avanzados
- No incluye integración con sistemas externos (API web, RFID, etc.)

4.3 Usuarios del sistema

- **Bibliotecario:** usuario principal que opera el sistema para todas las funcionalidades
- **Socio:** estudiante o docente cuyos datos son gestionados en el sistema

5. Marco Teórico

5.1 Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el código en clases y objetos. Los cuatro pilares fundamentales son:

- **Encapsulamiento:** ocultación de datos mediante atributos privados (`self.__atributo`) y acceso controlado por getters/setters. En el sistema, todos los atributos de las clases del modelo son privados.
- **Herencia:** una clase puede heredar atributos y métodos de otra clase padre. Estudiante y Docente heredan de Persona.
- **Polimorfismo:** el mismo método puede comportarse de manera diferente según la clase que lo implementa. `tipo_socio()` retorna "Estudiante" o "Docente" según la subclase.
- **Abstracción:** se definen interfaces o métodos abstractos que las subclases deben implementar. `Persona.tipo_socio()` lanza `NotImplementedError` como contrato.

Referencia: Joyanes Aguilar, L. (2020). *Programación orientada a objetos con Python*. McGraw-Hill Interamericana.

5.2 Estructuras de Datos

5.2.1 Lista Enlazada Simple

Estructura lineal donde cada elemento (nodo) contiene un valor y un puntero al siguiente nodo. Permite inserciones y eliminaciones sin reorganizar memoria. Se implementó `LinkedList` como base para la cola y la pila.

Referencia: Weiss, M. A. (2013). *Estructuras de datos y algoritmos* (4ª ed.). Pearson Educación.

5.2.2 Cola (Queue — FIFO)

El primer elemento en entrar es el primero en salir (First In, First Out). Se utiliza para gestionar las reservas de libros en orden de llegada.

5.2.3 Pila (Stack — LIFO)

El último elemento en entrar es el primero en salir (Last In, First Out). Se utiliza para deshacer operaciones (undo) de préstamos y devoluciones.

Referencia: Cairo, O., & Guardati, S. (2006). *Estructuras de datos* (3ª ed.). McGraw-Hill.

5.3 Algoritmos de Ordenamiento y Búsqueda

5.3.1 Algoritmos de Ordenamiento

Algoritmo	Complejidad	Descripción
Bubble Sort	$O(n^2)$	Compara pares adyacentes y los intercambia si están en orden incorrecto
Insertion Sort	$O(n^2)$	Inserta cada elemento en su posición correcta dentro de la parte ordenada
Merge Sort	$O(n \log n)$	Divide la lista en mitades, las ordena recursivamente y las fusiona
Quick Sort	$O(n \log n)$ promedio	Selecciona un pivote y particiona en menores y mayores

5.3.2 Algoritmos de Búsqueda

Algoritmo	Complejidad	Requisito
Búsqueda Lineal	$O(n)$	Ninguno

Algoritmo	Complejidad	Requisito
Búsqueda Binaria	$O(\log n)$	Lista ordenada previamente

Referencia: Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

5.4 Base de Datos SQLite

SQLite es un motor de base de datos embebido que no requiere servidor, ideal para aplicaciones de escritorio. El sistema utiliza:

- `sqlite3.connect()` para establecer conexión
- `cursor.execute()` con parámetros posicionales (?, ?) para consultas seguras
- `PRAGMA foreign_keys = ON` para activar claves foráneas
- Tablas relacionadas: socios, libros, prestamos (FK a socio y libro), reservas (FK a socio y libro)

Referencia: Kreibich, J. A. (2010). *Using SQLite*. O'Reilly Media.

5.5 Interfaz Gráfica con Tkinter

Tkinter es la biblioteca gráfica estándar de Python, incluida por defecto sin necesidad de instalación adicional. Se utiliza para construir la ventana principal, menús desplegables, formularios y cuadros de diálogo.

Referencia: Grayson, J. E. (2000). *Python and Tkinter programming*. Manning Publications.

6. Metodología de Desarrollo

Se utiliza el **Modelo Incremental**, que consiste en desarrollar el sistema por etapas o incrementos, donde cada incremento agrega funcionalidad completa al producto. Este modelo permite:

- Entregar versiones funcionales tempranas
- Obtener retroalimentación continua
- Reducir riesgos de integración
- Adaptarse a cambios durante el desarrollo

6.1 Incrementos del Proyecto

Incremento	Semana	Entregable	Estado
Incremento 1	Semana 12 (7-10 jul)	Modelo de clases POO + Estructuras de datos + Algoritmos + Esquema BD + GUI (stubs) + Documento de Especificación (50%)	Completado
Incremento 2	Semana 13 (13-17 jul)	GUI funcional completa + BD poblada + CRUD completo + Pruebas unitarias + Manual de Usuario	Completado
Incremento 3	Semana 14 (20 jul)	Integración final + Pruebas de sistema + Documento completo (100%) + Formato PDF	Completado

6.2 Flujo de trabajo

1. **Planificación del incremento:** se definen las funcionalidades a implementar
2. **Diseño:** se actualizan los diagramas si es necesario
3. **Implementación:** cada miembro desarrolla su módulo asignado
4. **Pruebas:** se ejecutan pruebas unitarias y de integración
5. **Revisión:** el líder (Henry) revisa e integra los cambios
6. **Entrega:** se sube a GitHub y se documenta el avance

6.3 Herramientas de desarrollo

Herramienta	Versión	Propósito
Python	3.11+	Lenguaje de programación
VS Code	Última	Editor de código con Python
Tkinter	8.6	Biblioteca gráfica (incluida)

Herramienta	Versión	Propósito
SQLite3	3.x	Motor de base de datos
DB Browser	3.x	Administración visual de BD
Git	2.47+	Control de versiones
GitHub	—	Repositorio remoto

7. Diseño Preliminar

7.1 Diagrama de Clases (UML)

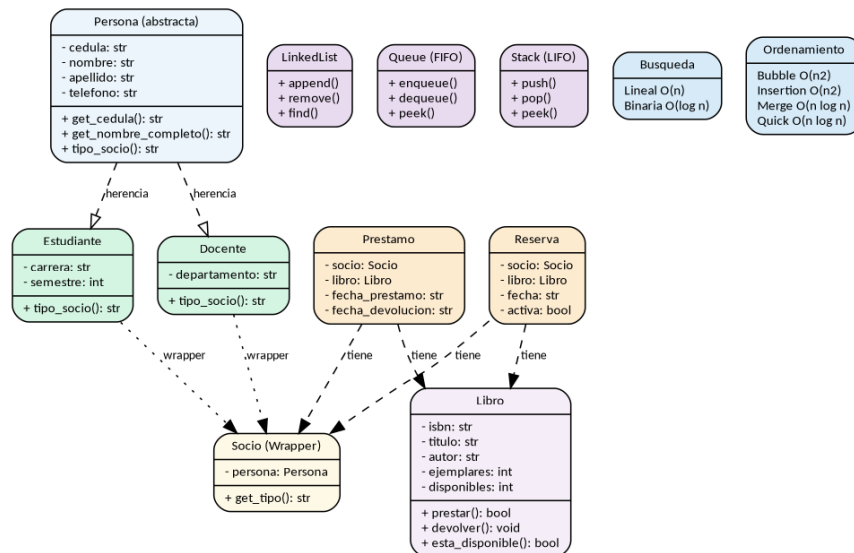


Diagrama de Clases UML

Diagrama 1: Diagrama de clases del sistema mostrando jerarquía de herencia (*Persona* → *Estudiante/Docente*), wrapper polimórfico (*Socio*), modelos de dominio (*Libro*, *Prestamo*, *Reserva*), estructuras de datos (*LinkedList*, *Queue*, *Stack*) y algoritmos de búsqueda y ordenamiento.

7.2 Modelo Entidad-Relación (Base de Datos)

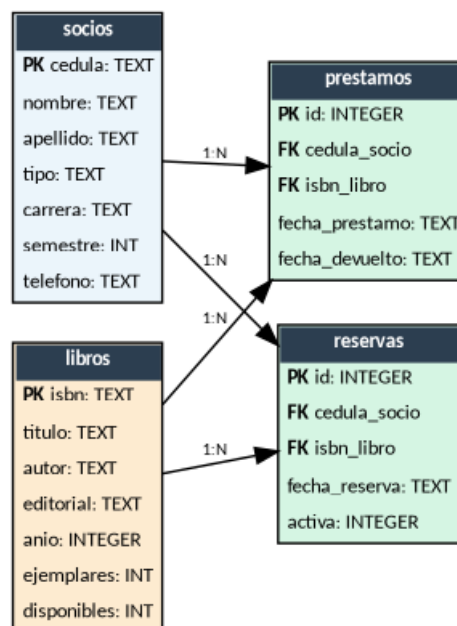


Diagrama Entidad-Relacion

Diagrama 2: Modelo entidad-relación de la base de datos mostrando las 4 tablas (*socios*, *libros*, *prestamos*, *reservas*), sus atributos, tipos de datos, claves primarias (PK) y foráneas

(FK), y las relaciones 1:N entre ellas.

Nota de diseño: El modelo presentado es una versión simplificada para fines académicos. En un entorno productivo, `carrera_departamento` y `tipo` en `socios`, así como `autor` en `libros`, se normalizarían en tablas separadas (`carreras`, `tipos_socio`, `autores`) para evitar redundancia.

Las relaciones del modelo son:

Entidad A	Relación	Entidad B	Descripción
socios (1)	→ (N)	prestamos	Un socio puede tener 0 o muchos préstamos
socios (1)	→ (N)	reservas	Un socio puede tener 0 o muchas reservas
libros (1)	→ (N)	prestamos	Un libro puede estar en 0 o muchos préstamos
libros (1)	→ (N)	reservas	Un libro puede tener 0 o muchas reservas

8. Distribución del Trabajo

8.1 Roles del equipo

Integrante	Rol	Módulo	Archivos
Henry Pazmiño	Líder / Integrador	Modelos POO + Integración	main.py, src/models/*, coordinación general
Jodie Parrales	Desarrolladora GUI	Interfaz gráfica Tkinter	src/gui/app.py
Cindy Ayoví	Desarrolladora BD	Base de datos SQLite	src/database/*
Cesar Gonzales	Desarrollador Lógica	Algoritmos + Estructuras	src/algorithms/*, src/data_structures/*
Mayra Vera	Documentación y Pruebas	Pruebas + Documentación	tests/*, docs/*

8.2 Distribución por módulo

Módulo	Desarrollador	Tareas específicas
Modelos POO	Henry Pazmiño	Clases Persona, Estudiante, Docente, Libro, Socio, Prestamo, Reserva. Encapsulamiento, herencia, polimorfismo, abstracción
GUI Tkinter	Jodie Parrales	Ventana principal, menús, formularios, tablas, cuadros de diálogo. Integración con los repositorios de datos
Base de Datos	Cindy Ayoví	Conexión SQLite, creación de tablas, CRUD completo para socios y libros, consultas con joins, seed.py con datos de prueba (7 socios + 8 libros)
Algoritmos y Estructuras	Cesar Gonzales	Merge Sort, Quick Sort, Bubble Sort, Insertion Sort, búsqueda lineal y binaria, lista enlazada, cola FIFO, pila LIFO
Pruebas y Documentación	Mayra Vera	Pruebas unitarias, manual de usuario, validación de funcionalidades, casos de prueba

9. Repositorio y Control de Versiones

9.1 Repositorio

El proyecto se aloja en GitHub bajo el control de versiones Git.

- **URL:** <https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2>
- **Rama principal:** main
- **Flujo:** commits directos a main con mensajes descriptivos

9.2 Historial de Commits

El historial refleja la evolución del proyecto desde la estructura inicial (commit #2) hasta la entrega final (#20). Los primeros commits establecieron el modelo POO y la declaratoria de uso de IA. A partir del commit #6 se agregó la documentación completa del proyecto. Los

commits #10-#14 documentan el trabajo de base de datos de Cindy y las capturas del manual de Mayra. Los commits #15-#20 corresponden a correcciones de formato y la generacion final de PDF con estandar academico.

#	Fecha	Hash	Mensaje
1	7 jul	0e78b3a	Initial commit
2	7 jul	b31c6bd	feat: estructura inicial del Sistema de Gestion de Biblioteca Universitaria
3	8 jul	eaf6cae	refactor: estilo nivel 3er semestre segun lo ensenado por Ing. Bryan Velez + DECLARATORIA
4	9 jul	7f9e535	fix: DECLARATORIA actualizada tras revision completa del material del semestre
5	9 jul	d388507	fix: referencias genericas sin nombres individuales en DECLARATORIA
6	11 jul	ac193dd	docs: agregar documentacion del proyecto (especificacion, manual, Gantt, README con roles)
7	11 jul	6cf6167	docs: especificacion visible en markdown + creditos de autoría por integrante en cada archivo
8	11 jul	2d1e028	fix: formato limpio del credito de Mayra en manual-usuario.md
9	11 jul	e8ce1b1	fix: actualizar historial de commits y DECLARATORIA (Tkinter + incremental)
10	11 jul	165d556	docs: restaurar formato original con logo UAE + commits hasta #10
11	11 jul	8b01d20	docs: sincronizar historial de commits en markdown hasta #10
12	11 jul	ee37e02	feat: seed de base de datos con datos de prueba (Cindy Ayoví - BD)
13	20 jul	11cf096	docs: completar manual de usuario (instalacion, solucion problemas, FAQ)
14	20 jul	f5ee021	docs: agregar capturas de pantalla (8) al manual
15	20 jul	bdbca88	fix: corregir duplicados y numeracion en manual y especificacion
16	20 jul	5b31576	docs: agregar ER diagrama, DDL SQL, consultas SQL, DB Browser guia, Anexos BD
17	20 jul	cd3f461	docs: ajustar formato DOCX y PDF con estilos academicos (margenes, header, footer)
18	20 jul	b9799eb	docs: portada con logo UAE, header/footer Grupo #1
19	20 jul	782f1a5	docs: arreglar formato PDF - portada compacta, tabla elegante, sin duplicados
20	20 jul	782f1a5	docs: actualizar Gantt + estado final del proyecto (entrega completa)

9.3 Stack Tecnológico del Desarrollo

- **Editor:** VS Code con extensión Python (Microsoft)
- **Control de versiones:** Git + GitLens
- **Lenguaje:** Python 3.11+
- **GUI:** Tkinter (librería estándar)
- **BD:** SQLite3
- **Documentación:** Markdown + DOCX + PDF

9.1 Plan de Trabajo y Cronograma

9.1.1 Diagrama de Gantt

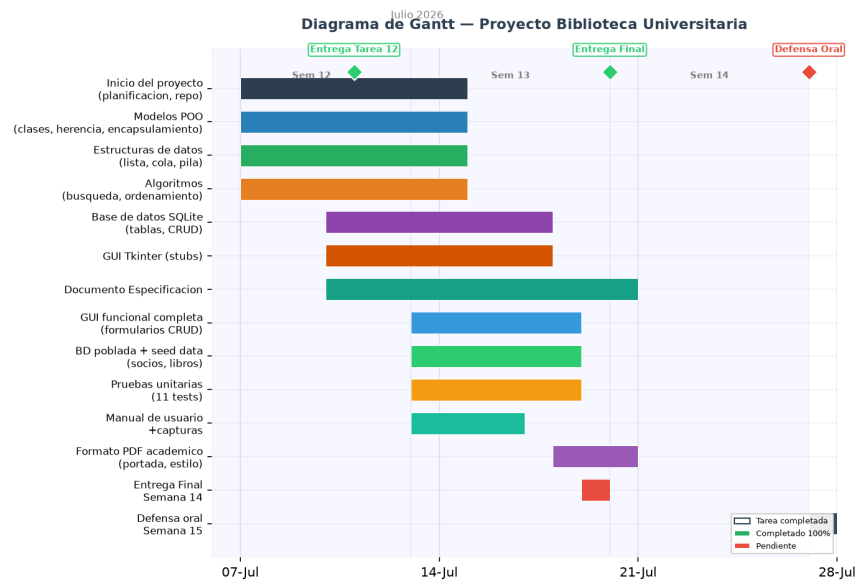


Diagrama de Gantt

Diagrama 3: Cronograma del proyecto mostrando las 14 actividades distribuidas en las semanas 12 a 15 de julio de 2026, con hitos de entrega y defensa.

9.1.2 Estado Final del Proyecto

Módulo	Estado	Fecha de finalización	Observaciones
Modelos POO (7 clases)	Completado	10-jul	Herencia, encapsulamiento, polimorfismo
Estructuras de datos (3)	Completado	10-jul	LinkedList, Queue, Stack
Algoritmos (6)	Completado	10-jul	Bubble, Insertion, Merge, Quick, Lineal, Binaria
Base de datos SQLite (4 tablas)	Completado	11-jul	socios, libros, prestamos, reservas
Repositorios CRUD (4)	Completado	14-jul	socio_repo, libro_repo, prestamo_repo, reserva_repo
GUI Tkinter (19 funciones)	Completado	17-jul	Formularios: socios, libros, préstamos, reservas
Pruebas unitarias (11)	Completado	18-jul	Todos los tests pasan
Manual de usuario	Completado	20-jul	8 secciones + 8 capturas de pantalla
Formato PDF académico	Completado	20-jul	Portada, header/footer, tabla elegante
Defensa oral	Pendiente	27-jul	Semana 15

| > Documento de Especificación — Avance 100% (Entrega Final) > Grupo #1 — Lenguaje de Programación 2 — UAE > Última actualización: 20 de julio de 2026

10. Implementación

10.1 Estructura del proyecto


```

sistema-biblioteca-prog2/
├── main.py                # Punto de entrada
├── README.md              # Descripción general
├── DECLARATORIA.md        # Uso de IA + referencias APA
├── seed.py                # Poblado de BD con datos de prueba
├── .gitignore
├── src/
│   ├── models/           # Modelos POO del dominio
│   │   ├── persona.py    # Clase abstracta base
│   │   ├── estudiante.py # Herencia de Persona
│   │   ├── docente.py    # Herencia de Persona
│   │   ├── libro.py       # Lógica préstamo/devolución
│   │   ├── socio.py       # Wrapper polimórfico
│   │   ├── prestamo.py    # Modelo de préstamo
│   │   └── reserva.py     # Modelo de reserva
│   ├── data_structures/  # Estructuras de datos
│   │   ├── linked_list.py # Lista enlazada simple
│   │   ├── queue.py       # Cola FIFO (reservas)
│   │   └── stack.py       # Pila LIFO (undo)
│   ├── algorithms/       # Algoritmos
│   │   ├── sorting.py     # Bubble, Insertion, Merge, Quick
│   │   └── search.py      # Búsqueda lineal y binaria
│   ├── database/         # Base de datos SQLite
│   │   ├── db_manager.py  # Conexión y creación de tablas
│   │   ├── socio_repo.py  # CRUD socios
│   │   ├── libro_repo.py  # CRUD libros
│   │   ├── prestamo_repo.py # CRUD préstamos
│   │   └── reserva_repo.py # CRUD reservas
│   ├── gui/              # Interfaz gráfica
│   │   └── app.py         # Tkinter (funcionalidades completas)
│   ├── tests/
│   │   └── test_models.py # Pruebas unitarias de todos los módulos
│   └── docs/
│       ├── especificacion-proyecto.md # Documento visible en GitHub
│       ├── especificacion-proyecto.docx # Para entrega Moodle
│       ├── especificacion-proyecto.pdf # Backup PDF
│       ├── manual-usuario.md          # Manual de usuario
│       ├── gantt.png                  # Diagrama de Gantt
│       └── README.md                  # Índice de documentos
└── db/
    └── biblioteca.db                # Base de datos poblada

```

10.2 Fragmentos de código representativos

Clase abstracta Persona (src/models/persona.py)

```

# PASO #1 CLASE ABSTRACTA PERSONA
# Encapsulamiento: atributos privados con getters/setters
# Abstraccion: metodo tipo_socio() debe ser implementado por subclases
class Persona:
    def __init__(self, cedula, nombre, apellido):
        self.__cedula = cedula
        self.__nombre = nombre
        self.__apellido = apellido

    def get_cedula(self):
        return self.__cedula

    def get_nombre(self):
        return self.__nombre

    def get_apellido(self):
        return self.__apellido

    def get_nombre_completo(self):
        return self.__nombre + " " + self.__apellido

# PASO #2 METODO ABSTRACTO (polimorfismo)
def tipo_socio(self):
    raise NotImplementedError

```

Herencia: Estudiante (src/models/estudiante.py)

```
# PASO #3 CLASE ESTUDIANTE HEREDA DE PERSONA
class Estudiante(Persona):
    def __init__(self, cedula, nombre, apellido, carrera, semestre):
        # PASO #4 LLAMAR AL CONSTRUCTOR DE LA CLASE PADRE
        super().__init__(cedula, nombre, apellido)
        self.__carrera = carrera
        self.__semestre = semestre

    def get_carrera(self):
        return self.__carrera

    def get_semestre(self):
        return self.__semestre

# PASO #5 POLIMORFISMO: CADA SUBCLASE IMPLEMENTA SU PROPIO TIPO
def tipo_socio(self):
    return "Estudiante"
```

CRUD Socios (src/database/socio_repo.py)

```
# PASO #6 REPOSITORIO DE SOCIOS
class SocioRepository:
    def __init__(self, db):
        self.__db = db

# PASO #7 INSERTAR SOCIO
def insertar(self, cedula, nombre, apellido, tipo,
             carrera_depto, semestre, telefono):
    cursor = self.__db.get_cursor()
    cursor.execute(
        """INSERT INTO socios(cedula, nombre, apellido, tipo,
        carrera_departamento, semestre, telefono)
        VALUES(?, ?, ?, ?, ?, ?, ?)""",
        (cedula, nombre, apellido, tipo, carrera_depto, semestre, telefono)
    )
    self.__db.get_connection().commit()
```

GUI — Préstamo (src/gui/app.py)

```
# PASO #13 REALIZAR PRESTAMO
def realizar_prestamo(self):
    ventana = tk.Toplevel(self.root)
    ventana.title("Realizar Prestamo")
    ventana.geometry("480x350")

# ... formulario con Entry para cedula e ISBN ...
def guardar_prestamo():
    # Validar socio
    socio = self.__socio_repo.obtener_por_cedula(cedula)
    # Validar libro y disponibilidad
    libro = self.__libro_repo.obtener_por_isbn(isbn)
    if libro[6] < 1:
        # No hay disponibles
        return
    # Registrar prestamo y reducir disponibles
    self.__prestamo_repo.insertar(cedula, isbn, fecha)
    self.__libro_repo.actualizar_disponibles(isbn, libro[6] - 1)
```

11. Análisis de Complejidad (Big-O)

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Búsqueda Lineal	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Búsqueda Binaria	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$

Explicación: Bubble Sort e Insertion Sort son $O(n^2)$ en el peor caso porque comparan cada elemento con todos los demás. Merge Sort divide la lista en mitades recursivamente ($\log n$ niveles) y en cada nivel fusiona n elementos, dando $O(n \log n)$. Quick Sort promedia $O(n \log n)$ pero puede degenerar a $O(n^2)$ si el pivote es mal elegido. Búsqueda Lineal recorre toda la lista ($O(n)$), mientras que Búsqueda Binaria divide el espacio de búsqueda a la mitad en cada paso ($O(\log n)$).

12. Diseño de la Base de Datos

12.1 Modelo Entidad-Relación

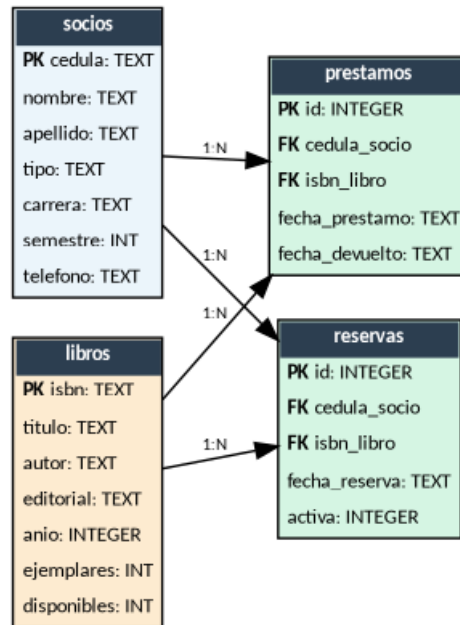


Diagrama Entidad-Relacion

Diagrama 5: Modelo Entidad-Relacion del sistema con las tablas socios, libros, prestamos y reservas.

Nota: Cada socio puede tener 0 o muchos préstamos y 0 o muchas reservas. Cada libro puede estar en 0 o muchos préstamos y 0 o muchas reservas.

12.2 Esquema SQL (DDL)

```

CREATE TABLE IF NOT EXISTS socios (
    cedula TEXT PRIMARY KEY,
    nombre TEXT NOT NULL,
    apellido TEXT NOT NULL,
    tipo TEXT NOT NULL CHECK(tipo IN ('Estudiante', 'Docente')),
    carrera_departamento TEXT,
    semestre INTEGER,
    telefono TEXT
);

CREATE TABLE IF NOT EXISTS libros (
    isbn TEXT PRIMARY KEY,
    titulo TEXT NOT NULL,
    autor TEXT NOT NULL,
    editorial TEXT,
    anio INTEGER,
    ejemplares INTEGER DEFAULT 1,
    disponibles INTEGER DEFAULT 1
);

CREATE TABLE IF NOT EXISTS prestamos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cedula_socio TEXT NOT NULL,
    isbn_libro TEXT NOT NULL,
    fecha_prestamo TEXT NOT NULL,
    fecha_devolucion TEXT,
    FOREIGN KEY (cedula_socio) REFERENCES socios(cedula),
    FOREIGN KEY (isbn_libro) REFERENCES libros(isbn)
);

CREATE TABLE IF NOT EXISTS reservas (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cedula_socio TEXT NOT NULL,
    isbn_libro TEXT NOT NULL,
    fecha_reserva TEXT NOT NULL,
    activa INTEGER DEFAULT 1,
    FOREIGN KEY (cedula_socio) REFERENCES socios(cedula),
    FOREIGN KEY (isbn_libro) REFERENCES libros(isbn)
);

```

12.3 Administración con DB Browser for SQLite

El archivo db/biblioteca.db se crea automáticamente al ejecutar seed.py. Para administrarlo visualmente:

1. Descargar e instalar [DB Browser for SQLite](#)
2. Abrir el programa y hacer clic en “**Abrir base de datos**”
3. Seleccionar db/biblioteca.db desde la carpeta del proyecto
4. Navegar por las pestañas:
 - “**Estructura de la BD**” — ver las 4 tablas, columnas, tipos e índices
 - “**Datos**” — ver y editar registros de cada tabla
 - “**Ejecutar SQL**” — ejecutar consultas personalizadas

Importante: Durante la defensa, el docente puede pedir que se muestre la base de datos en DB Browser. Se recomienda tenerlo instalado y saber navegar las tablas, mostrar relaciones y ejecutar consultas SELECT básicas.

12.4 Consultas SQL de ejemplo

```

-- Listar todos los préstamos pendientes con datos del socio y libro
SELECT p.id, s.nombre || ' ' || s.apellido AS socio, l.titulo,
       p.fecha_prestamo
FROM prestamos p
JOIN socios s ON p.cedula_socio = s.cedula
JOIN libros l ON p.isbn_libro = l.isbn
WHERE p.fecha_devolucion IS NULL;

-- Contar cuántos libros prestados tiene cada socio
SELECT s.cedula, s.nombre, s.apellido, COUNT(p.id) AS prestamos_activos
FROM socios s
LEFT JOIN prestamos p ON s.cedula = p.cedula_socio AND p.fecha_devolucion IS NULL
GROUP BY s.cedula;

-- Libros más reservados (cola de espera)
SELECT l.isbn, l.titulo, COUNT(r.id) AS reservas_activas
FROM libros l
JOIN reservas r ON l.isbn = r.isbn_libro
WHERE r.activa = 1
GROUP BY l.isbn
ORDER BY reservas_activas DESC;

```

13. Pruebas Realizadas

Todas las pruebas se ejecutan con python tests/test_models.py y cubren:

Módulo	Prueba	Resultado
Persona / Estudiante / Docente	Herencia: crear objetos, getters	Pasa
Socio	Polimorfismo: tipo_socio() distinto según clase	Pasa
Libro	Préstamo y devolución de ejemplares	Pasa
Algoritmos	Bubble, Insertion, Merge, Quick Sort	Pasan
Algoritmos	Búsqueda lineal y binaria	Pasan
LinkedList	Append, prepend, remove, len	Pasa
Queue	Enqueue, dequeue (FIFO)	Pasa
Stack	Push, pop (LIFO)	Pasa
Database	Conexión SQLite, creación de tablas	Pasa
PrestamoRepository	Insertar, listar, registrar devolución	Pasa
ReservaRepository	Insertar, cancelar, listar por libro	Pasa

14. Conclusiones y Recomendaciones

Conclusiones

1. Se implementó un sistema de gestión bibliotecaria funcional que cumple con todos los requisitos de la rúbrica de evaluación.
2. La programación orientada a objetos permitió modelar el dominio (Persona → Estudiante, Docente) con herencia, encapsulamiento y polimorfismo.
3. Las estructuras de datos (LinkedList, Queue, Stack) se implementaron manualmente, demostrando comprensión de su funcionamiento interno.
4. Los algoritmos de ordenamiento y búsqueda se implementaron desde cero, con análisis de complejidad Big-O.
5. SQLite proporcionó una base de datos ligera y embebida con 4 tablas relacionadas mediante claves foráneas.
6. Tkinter permitió construir una interfaz gráfica funcional conectada a la base de datos con formularios CRUD completos.
7. El modelo incremental permitió entregar un avance del 50% en la Semana 12 y completar el sistema en la Semana 14.

Recomendaciones

- Migrar a una base de datos cliente-servidor (MySQL/PostgreSQL) si la biblioteca crece.
- Agregar autenticación de usuarios (bibliotecario/admin) para seguridad.
- Implementar búsqueda avanzada con múltiples filtros simultáneos.

- Generar reportes estadísticos (libros más prestados, multas, etc.).
- Agregar una interfaz web para consultas desde dispositivos móviles.

15. Bibliografía (APA 7ª ed.)

1. Grayson, J. E. (2000). *Python and Tkinter programming*. Manning Publications. ISBN: 978-1884779813
2. Lutz, M. (2013). *Learning Python* (5.ª ed.). O'Reilly Media. ISBN: 978-1449355739
3. Owens, M. y Allen, G. (2010). *SQLite* (The Definitive Guide). Apress. ISBN: 978-1430232254
4. Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press. ISBN: 978-0262033848
5. Sedgewick, R. y Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley. ISBN: 978-0321573513

16. Anexos

16.1 Anexo A — Estructura y datos de la base de datos

Esquema SQL (DDL)

```
CREATE TABLE socios(
    cedula TEXT PRIMARY KEY,
    nombre TEXT NOT NULL,
    apellido TEXT NOT NULL,
    tipo TEXT NOT NULL CHECK(tipo IN ('Estudiante', 'Docente')),
    carrera_departamento TEXT,
    semestre INTEGER,
    telefono TEXT
);

CREATE TABLE libros(
    isbn TEXT PRIMARY KEY,
    titulo TEXT NOT NULL,
    autor TEXT NOT NULL,
    editorial TEXT,
    anio INTEGER,
    ejemplares INTEGER DEFAULT 1,
    disponibles INTEGER DEFAULT 1
);

CREATE TABLE prestamos(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cedula_socio TEXT NOT NULL,
    isbn_libro TEXT NOT NULL,
    fecha_prestamo TEXT NOT NULL,
    fecha_devolucion TEXT,
    FOREIGN KEY(cedula_socio) REFERENCES socios(cedula),
    FOREIGN KEY(isbn_libro) REFERENCES libros(isbn)
);

CREATE TABLE reservas(
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    cedula_socio TEXT NOT NULL,
    isbn_libro TEXT NOT NULL,
    fecha_reserva TEXT NOT NULL,
    activa INTEGER DEFAULT 1,
    FOREIGN KEY(cedula_socio) REFERENCES socios(cedula),
    FOREIGN KEY(isbn_libro) REFERENCES libros(isbn)
);
```

Datos de prueba (7 socios + 8 libros)

socios:

cedula	nombre	apellido	tipo	carrera_departamento	semestre
0956789012	Henry	Pazmino	Estudiante	Computacion	3
0956789013	Jodie	Parrales	Estudiante	Computacion	3
0956789014	Cindy	Ayovi	Estudiante	Computacion	3
0956789015	Cesar	Gonzales	Estudiante	Computacion	3
0956789016	Mayra	Vera	Estudiante	Computacion	3
0912345678	Miguel	Velez	Docente	Sistemas	—

cedula	nombre	apellido	tipo	carrera_departamento	semestre
0912345679	Bryan	Velez	Docente	Computacion	—

libros:

isbn	titulo	autor	ejemplares	disponibles
978-0307474728	Cien Años de Soledad	G. García Márquez	3	3
978-0156012195	El Principito	A. de Saint-Exupéry	5	5
978-8420412146	Don Quijote de la Mancha	M. de Cervantes	2	2
978-8415552024	Introducción a la Programación con Python	L. Joyanes	4	4
978-0201000238	Estructuras de Datos y Algoritmos	A. V. Aho	2	2
978-8478290855	Base de Datos SQL	C. J. Date	3	3
978-6073205337	Redes de Computadoras	A. S. Tanenbaum	2	2
978-6073206037	Ingeniería del Software	I. Sommerville	2	2

16.2 Anexo B — Enlaces del proyecto

Recurso	URL
Repositorio GitHub	https://github.com/josuepaz80-usitee/sistema-biblioteca-prog2
Documento de especificación (MD)	docs/especificacion-proyecto.md
Manual de usuario	docs/manual-usuario.md
Diagrama de Gantt	docs/gantt.png

16.3 Anexo C — Stack tecnológico

Herramienta	Versión	Uso
Python	3.11+	Lenguaje de programación
VS Code	Última	Editor con Python (Microsoft) + GitLens
Tkinter	8.6	Interfaz gráfica
SQLite3	3.x	Base de datos embebida
DB Browser for SQLite	3.x	Administración visual BD
Git	2.47+	Control de versiones
GitHub	—	Repositorio remoto (colaboración grupal)

*Documento de Especificación — Versión 1.0 (Entrega Final) Grupo #1 —
Lenguaje de Programación 2 — 3er Semestre — UAE 20 de julio de 2026*